MULTI-AGENT SYSTEMS

Case Study Handbook

Architecture, Design Patterns, and Decision Frameworks — with Financial Services and SaaS Case Studies

A Learning Companion

CodeSentinel · RByte.ai

Contents

1. How This Handbook Works

This handbook walks through multi-agent systems using case studies from financial services and SaaS — industries where these systems are already running in production. Read it at your own pace. Each case study can stand alone, but they build on each other, so reading in order will make later sections easier to follow.

Every case study includes two short boxes:

> **WHY IT FITS**
> Explains why this architecture or pattern is the right choice for the scenario.

> **WHERE IT BREAKS**
> Explains the specific limitation or failure mode — every pattern has trade-offs, and it's worth knowing them upfront.

One distinction to keep in mind throughout

- Architecture is the structural blueprint — who holds control. It's either Centralized, Decentralized, or Hybrid.
- Design Pattern is the interaction style within that structure — Manager–Worker, Peer–Swarm, Federated Teams, or Sequential Pipeline.

These are two different questions about the same system. Case Studies 1 and 2 are revisited later in the handbook — first through the architecture lens, then again through the pattern lens — so you can see both questions answered for the same real system.

2. Architecture Layer — Three Case Studies

Architecture answers one question: where does control live? The three case studies below show Centralized, Decentralized, and Hybrid architectures, each through a real industry scenario.

ARCHITECTURE — CENTRALIZED

Case Study 1 — SaaS Customer Support Triage

Industry	SaaS — customer support platform, roughly 50,000 tickets per month
Architecture	Centralized — a single Manager agent holds full control
Core tension	Speed and consistency versus a single point of failure

The Scenario

A mid-size SaaS company receives support tickets spanning billing, technical issues, and account or returns queries. Leadership wants consistent responses, a full audit trail for compliance, and predictable cost per ticket. Every incoming ticket is read first by a single Manager agent, which classifies it and hands off to the right specialist.

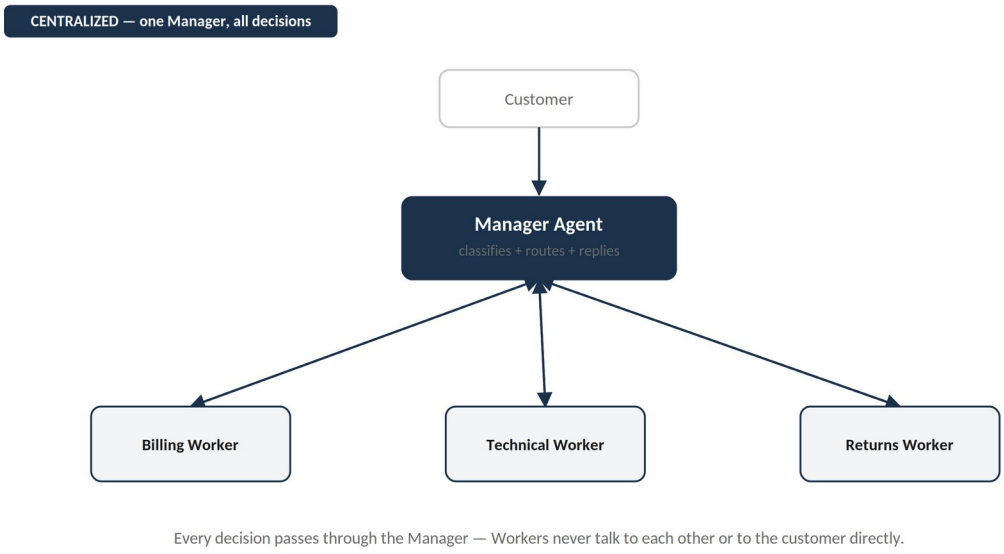


Figure 2.1 — Centralized architecture: every decision flows through one Manager agent.

Agent Roster

Agent	Role	Inputs	Outputs
Manager Agent	Classifies ticket, routes, consolidates final reply	Raw ticket text, customer metadata	Routed task and final customer response
Billing Worker	Resolves invoice and payment queries	Billing sub-task from Manager	Draft resolution to Manager
Technical Worker	Diagnoses product and	Technical sub-task from	Draft resolution to Manager

Agent	Role	Inputs	Outputs
	technical issues	Manager	
Returns Worker	Handles refund and return eligibility	Returns sub-task from Manager	Draft resolution to Manager

How It Works, Step by Step

1. The ticket arrives and is read in full by the Manager agent.
2. The Manager classifies the intent — billing, technical, or returns — and may split one ticket into several sub-tasks.
3. The Manager delegates each sub-task to the right Worker, passing only the context that Worker needs.
4. Each Worker returns a draft resolution to the Manager — never directly to the customer.
5. The Manager combines all Worker outputs into one reply, checks tone and compliance, and sends the final response.
6. The Manager logs the full decision trail — classification, delegation, consolidation — for audit purposes.

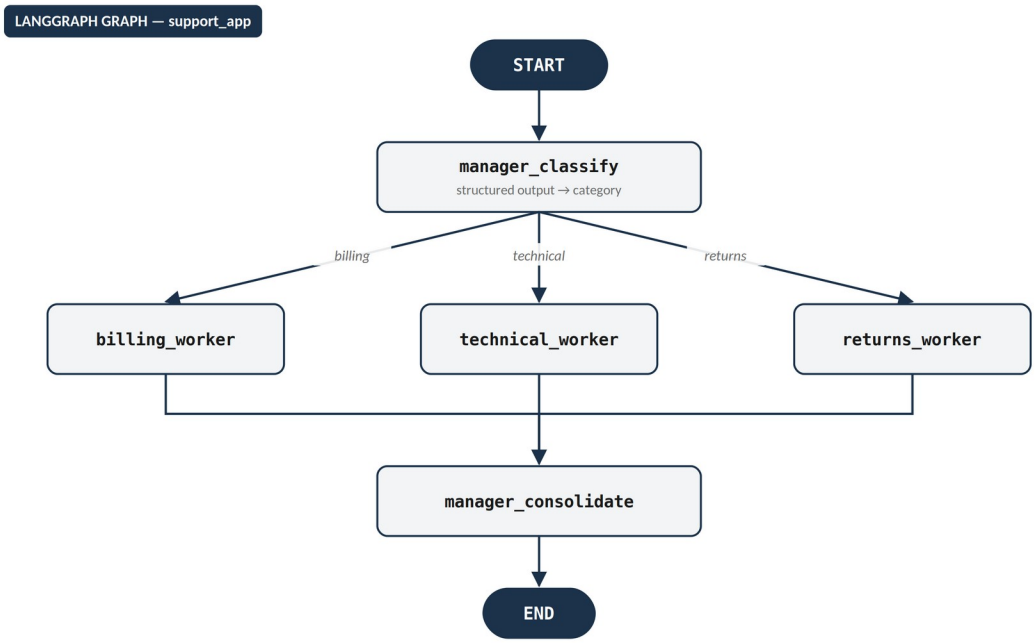


Figure 2.1b — The LangGraph implementation of this case study: manager_classify routes to exactly one worker node, which always returns through manager_consolidate.

WHY IT FITS

Every decision passes through one node, so the audit trail is simple, tone and compliance stay consistent across replies, and cost per ticket is predictable because the Manager can be monitored and rate-limited centrally.

WHERE IT BREAKS

At high ticket volume the Manager becomes a bottleneck — every ticket, however simple, waits on the same node. If the Manager fails or is misconfigured, the entire pipeline stops, with no graceful fallback.

Questions to Think About

latency, 423 errors

1. *At what ticket volume would you expect the Manager to become a bottleneck? What metric would tell you first?*
2. *Would you ever let a Worker respond directly to a customer, skipping the Manager? What would that cost you?*
3. *How would you add backup for the Manager agent without turning this into a Decentralized system?*

adding new instances of manager

The Manager becomes a bottleneck when its processing capacity is lower than incoming ticket volume,
which would first appear as increasing queue depth and rising latency.
Workers can bypass the Manager for low-risk tasks to reduce latency, but this sacrifices consistency, governance, and quality control.
To add resilience while keeping the system centralized,
I would run multiple Manager instances behind a load balancer or failover mechanism, with one acting as the coordinator and others serving as backups.

ARCHITECTURE — DECENTRALIZED

Case Study 2 — Algorithmic Trading Across Multiple Exchanges

Industry	Financial services — a quantitative trading firm running bots on six exchanges
Architecture	Decentralized — no central controller; each bot acts independently
Core tension	Microsecond speed versus firm-wide visibility into risk

The Scenario

A trading firm runs automated trading bots — one per exchange — executing arbitrage and market-making strategies. Sending every decision through a central risk engine would add delay measured in milliseconds, which is too slow for these strategies to work. So each bot runs independently, watching only its own exchange's order book and reacting instantly, without waiting for permission from any central brain.

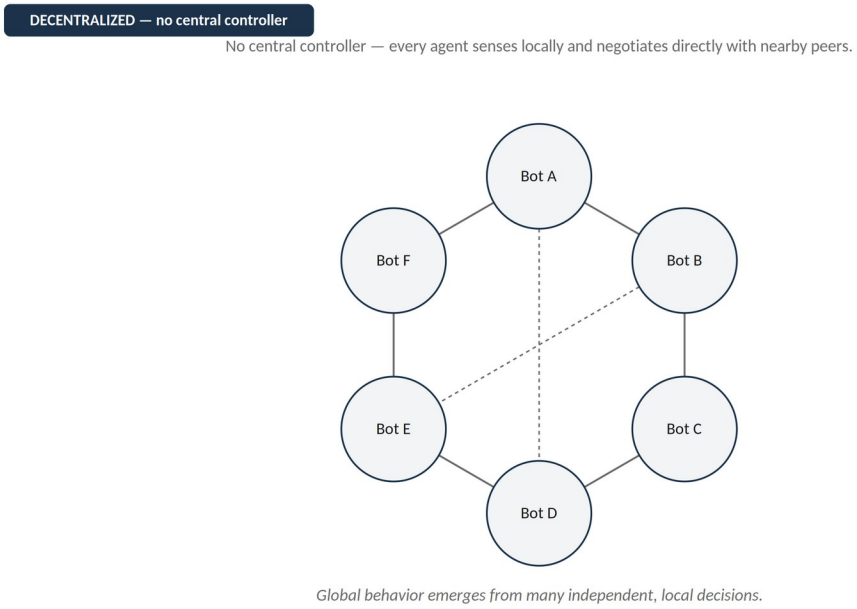


Figure 2.2 — Decentralized architecture: no central node; every bot senses and acts locally.

Agent Roster

Agent	Role	Senses	Reports To
Exchange Trading Bot (×6)	Executes buy or sell orders within its own risk limits	Its own exchange's live order book	No central node — acts independently
Arbitrage Scanner (paired with each bot)	Flags a price difference worth trading on	Prices on a few nearby exchanges	Its own paired Trading Bot only
Regional Risk Monitor (×3)	Can pause a bot if regional exposure crosses a limit	Combined exposure of a handful of nearby bots	Not a central controller — regional scope only

How It Works, Step by Step

1. Each Trading Bot continuously reads its own exchange's live order book — price, volume, spread — with no view of the other five exchanges.

2. Its paired Arbitrage Scanner compares visible prices with a few nearby exchanges and flags a profitable opportunity.

3. The Trading Bot executes the trade instantly if it's profitable and within its own pre-set risk limits — no permission needed from any central node.

4. A Regional Risk Monitor tracks the combined exposure of a handful of nearby bots and can pause any one of them if regional exposure crosses a set limit.

5. No single agent ever sees the firm's total global exposure in real time — that full picture is only reconstructed later from combined logs.

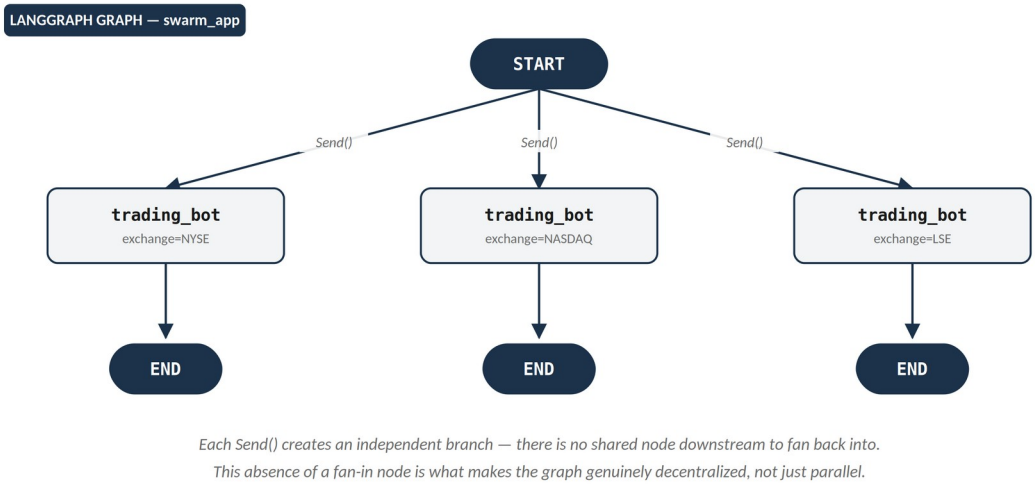


Figure 2.2b — The LangGraph implementation: Send() dispatches independent branches with no shared node downstream — the absence of a fan-in is what makes this decentralized.

> **WHY IT FITS**
> Speed is the entire value of these strategies — even a few milliseconds of round-trip to a central controller destroys the edge. Decentralization also means one exchange's outage or one bot's failure doesn't stop trading everywhere else.

> **WHERE IT BREAKS**
> No bot can see the firm's total exposure in real time, so several bots can unknowingly build correlated risk at the same moment. A bad trading day gets pieced together afterward from separate logs — it isn't caught as it happens.

Questions to Think About

4. *What is the smallest piece of shared, real-time information you'd want every bot to see — and would adding it turn this into a Hybrid system?*
5. *How would this firm catch correlated risk building up across bots without slowing any single bot down?*
6. *Compare this failure mode to Case Study 1's Manager: which one is easier to explain to a regulator after the fact?*

4. What is the smallest piece of shared, real-time information you'd want every bot to see — and would adding it turn this into a Hybrid system?
The smallest useful shared information would be a global risk indicator, such as total exposure, risk utilization, or emergency risk alerts. This allows each bot to remain autonomous while avoiding actions that collectively create excessive risk. Yes, adding even a minimal shared state would move the architecture toward a Hybrid system, sinc
5. How would this firm catch correlated risk building up across bots without slowing any single bot down?
The firm could use asynchronous monitoring. Each bot would publish its positions and decisions to a shared event stream, while a separate monitoring service continuously aggregates and analyzes the data for concentration or correlation risks. Since monitoring occurs in parallel, individual bots do not need to wait for approval, preserving s
6. Compare this failure mode to Case Study 1's Manager: which one is easier to explain to a regulator after the fact?
The Manager-based system is generally easier to explain because there is a clear chain of responsibility and a centralized decision record. In a decentralized system, risk may emerge from many independent decisions, making it harder to identify a single cause or responsible agent. Although decentralized systems can be more scalable an

ARCHITECTURE — HYBRID

Case Study 3 — Digital Loan Origination Platform

Industry	Fintech lending — a digital platform originating personal loans
Architecture	Hybrid — a central Loan Manager with internally decentralized specialist teams
Core tension	One auditable decision versus specialist team autonomy

The Scenario

A digital lending platform processes loan applications end to end. Every application needs one clear, auditable decision — approved, declined, or needs more information — which is a regulatory requirement. But the actual work behind that decision spans very different specialties: checking credit history, verifying documents, and, if approved, disbursing funds. A central Loan Manager owns the applicant-facing decision and the compliance record, without micromanaging how each team does its internal work.

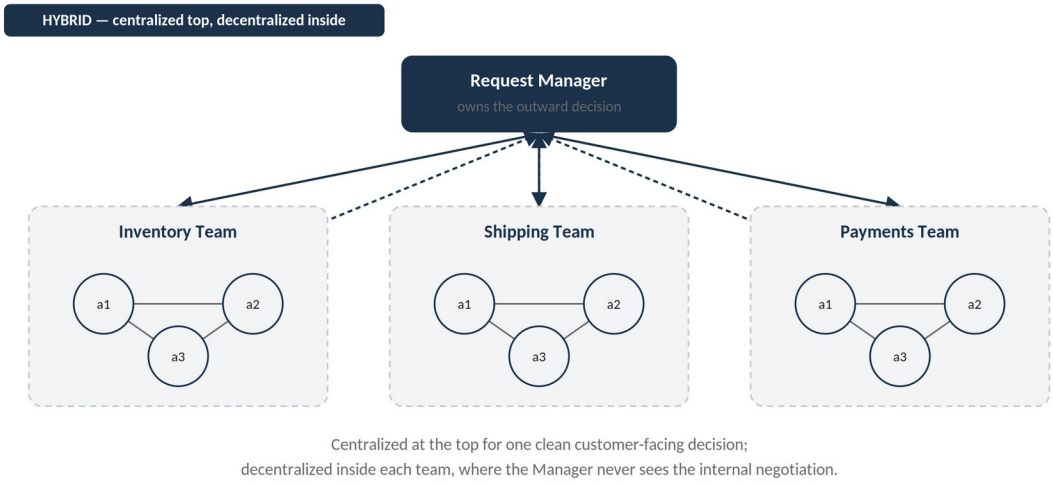


Figure 2.3 — Hybrid architecture: one central owner at the top, decentralized teams underneath.

Agent Roster

Agent / Team	Role	Control Style
Loan Manager	Owns the application lifecycle, requests team input, issues the final decision, keeps the compliance record	Centralized, top level
Credit Underwriting Team (3 agents)	Credit bureau check, income verification, risk scoring	Decentralized within team
Document Verification Team (2 agents)	ID verification, address proof matching	Decentralized within team
Disbursement Team (2 agents)	Bank account validation, fund transfer, confirmation	Decentralized within team

How It Works, Step by Step

1. The Loan Manager receives the application and sends parallel requests to Underwriting and Document Verification.
2. Inside the Underwriting Team, agents negotiate internally on a combined risk score — the Loan Manager only sees the final score, not the internal back-and-forth.
3. The Document Verification Team cross-checks ID and address internally and flags any mismatch.
4. Each team reports one consolidated result to the Loan Manager, not raw internal detail.
5. The Loan Manager combines both results into one decision: approve, decline, or request more information.
6. If approved, the Loan Manager triggers the Disbursement Team, which handles the fund transfer internally and reports back once complete.
7. The Loan Manager logs the entire decision trail for regulatory audit.

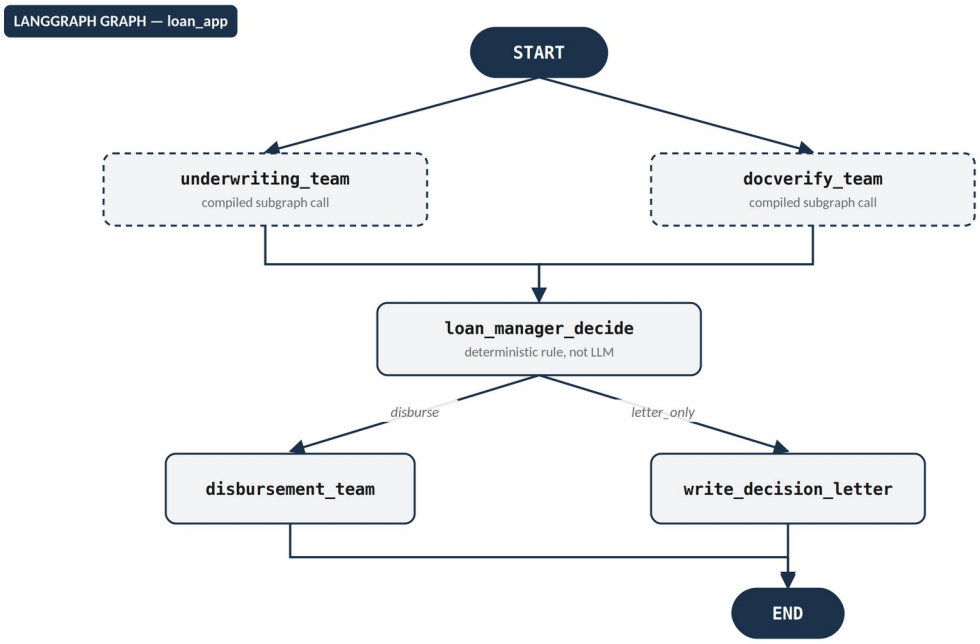


Figure 2.3b — The LangGraph implementation: underwriting_team and docverify_team are each a compiled subgraph called as one node from the top-level Loan Manager graph.

> **WHY IT FITS**
> Lending decisions need one centrally owned, auditable record — but forcing every internal credit-scoring negotiation through that same Manager would slow down underwriting for no real benefit. Hybrid gives one clean decision record with flexible internal execution.

> **WHERE IT BREAKS**
> The most common problem in a system like this is unclear ownership at the boundary — for example, who gets told first if Document Verification flags a mismatch while Underwriting is still working? Getting the boundary wrong recreates a bottleneck or an audit gap inside the wrong layer.

Questions to Think About

7. *If Document Verification flags a mismatch mid-underwriting, who should be told first, and why?*
8. *What is the minimum information Underwriting must report upward, versus keep internal?*

9. *Would you nest Disbursement inside the Loan Manager directly, or keep it a fourth independent team? Why?*

7. If Document Verification flags a mismatch mid-underwriting, who should be told first, and why?

Answer:
Document Verification should notify the Underwriting Agent first because underwriting owns risk assessment and loan approval decisions.
Underwriting can review the issue, request clarification, and escalate to the Loan Manager only if necessary.

Flow:
Document Verification → Underwriting → Loan Manager

8. What is the minimum information Underwriting must report upward, versus keep internal?

Report Upward:
- Application ID
- Approval/Rejected/Pending status
- Risk level
- Key decision reasons
- Required actions/escalations

Keep Internal:
- Detailed risk calculations
- Credit analysis
- Supporting evidence
- Internal notes
- Verification discussions

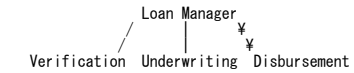
Principle:
Escalate conclusions, not detailed working data.

9. Would you nest Disbursement inside the Loan Manager directly, or keep it a fourth independent team? Why?

Answer:
Keep Disbursement as an independent agent/team.

Reasons:
- Clear separation of responsibilities
- Easier scaling and maintenance
- Supports independent integrations (banks, payment gateways, fraud checks)
- Keeps the Loan Manager focused on orchestration

Architecture:



The Loan Manager coordinates all agents while each specialized agent handles its own domain.

3. Design Pattern Layer — Four Case Studies

Design pattern answers a different question than architecture: given a structure, how do agents actually interact? The first two case studies below revisit Case Studies 1 and 2 through this new lens — same scenario, different question. The third is new. The fourth is a familiar shape seen through a Sequential Pipeline.

PATTERN — MANAGER AND WORKER

Revisiting Customer Support Triage

Same system as Case Study 1. The architecture question was who controls whom (Centralized). The pattern question is how the Manager actually interacts with Workers, moment to moment.

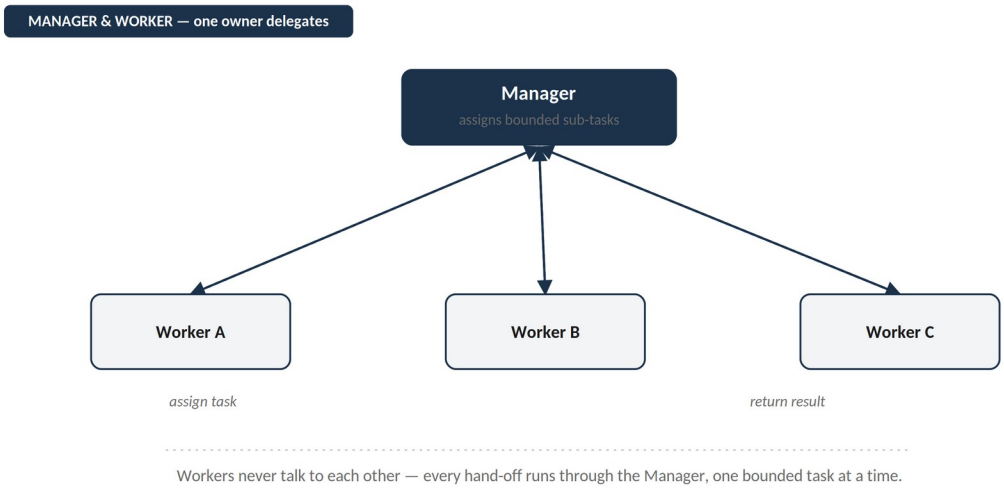


Figure 3.1 — Manager and Worker: bounded request and response, no Worker-to-Worker contact.

What Changes Under the Pattern Lens

- Interaction is strictly request and response: the Manager assigns a bounded sub-task, the Worker returns exactly one result, with no back-and-forth.
- Workers never talk to each other — all coordination happens through the Manager, even when two Workers' outputs are related.
- This pattern isn't tied to one architecture. Manager–Worker can run inside a Decentralized team too, such as within Case Study 3's Underwriting team — pattern and architecture are independent choices.

> **WHY IT FITS**
> Clear task splitting with no need for Workers to share context with each other. This is the easiest pattern to reason about, test, and log — each Worker behaves like a simple function from the Manager's point of view.

> **WHERE IT BREAKS**
> If two Workers' outputs are actually connected — Billing and Returns both needing to agree on a refund amount, for instance — the Manager has to run them one after another, or the pattern quietly turns into extra back-and-forth through the Manager, adding the delay the pattern was meant to avoid.

Questions to Think About

10. *Name a case where two Workers here would genuinely need each other's output. How would you handle it without breaking the pattern?*
11. *Is Manager–Worker the same thing as Centralized architecture, or just the most common pattern used inside one?*

10. Name a case where two Workers would genuinely need each other's output. How would you handle it without breaking the pattern?

Example:
In a loan-processing system, the Risk Assessment Worker calculates a risk score, while the Eligibility Worker determines whether the applicant meets lending criteria. The Eligibility Worker may need the risk score before making a decision.

How to handle it without breaking the Manager–Worker pattern:
1. Manager sends task to Risk Assessment Worker.
2. Risk Assessment Worker returns the risk score.
3. Manager includes the risk score in a new request to the Eligibility Worker.
4. Eligibility Worker returns the eligibility decision.

Workers never communicate directly. The Manager coordinates all information flow.

11. Is Manager–Worker the same thing as Centralized architecture, or just the most common pattern used inside one?

Answer:
Manager–Worker is a communication pattern, not an architecture.

Centralized architecture describes who has authority and control (a central coordinator manages other agents).

Manager–Worker describes how agents interact (manager assigns tasks, workers return results, no worker-to-worker communication).

A Centralized architecture often uses the Manager–Worker pattern, but the pattern can also exist within a Decentralized or Hybrid architecture as part of a larger system.

Key distinction:
- Architecture = who controls whom.
- Pattern = how they communicate and collaborate.

PATTERN — PEER AND SWARM

Revisiting Algorithmic Trading Across Exchanges

Same system as Case Study 2. The architecture question was the absence of a central controller (Decentralized). The pattern question is what the interaction between bots actually looks like.

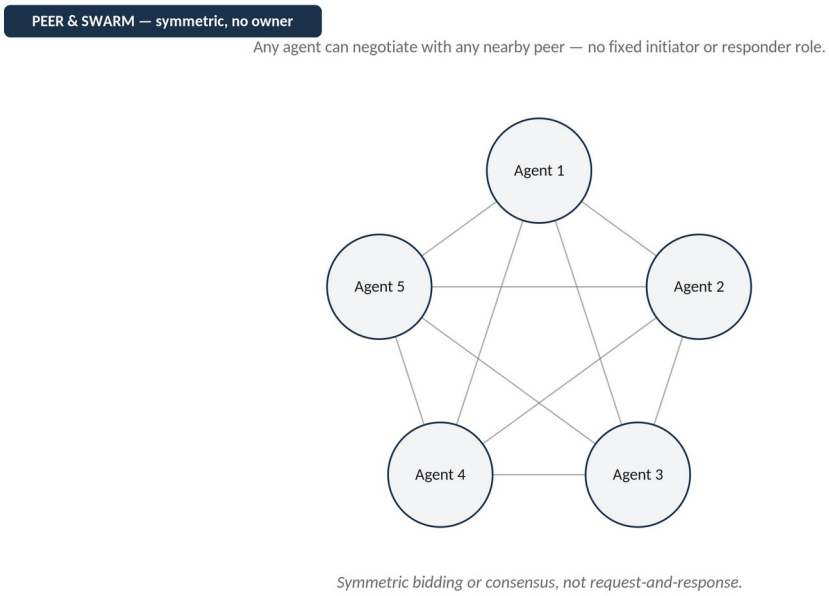


Figure 3.2 — Peer and Swarm: symmetric, independent action with no fixed roles.

What Changes Under the Pattern Lens

- Interaction is symmetric — any bot can act the moment it sees an opportunity; there's no fixed initiator or responder role like in Manager–Worker.
- There's no negotiation protocol between bots the way there is in a true auction — each bot reacts to shared market prices rather than talking to other bots directly.
- Firm-wide behavior is emergent: no single bot's decision produces firm-wide stability, only the sum of every bot's independent decisions does.

> **WHY IT FITS**
> A fast-moving market with strict latency limits makes a fixed role (who decides right now?) impossible to maintain — independent, symmetric action avoids that overhead entirely.

> **WHERE IT BREAKS**
> Nothing guarantees the bots' independent actions add up to something healthy for the firm. Several bots reacting to the same market signal at the same time can create a pile-up of correlated risk that no single bot intended and no single bot can see.

Questions to Think About

12. *How would you catch a correlated pile-up like this without adding a central observer, which would compromise the pattern?*
13. *What is the smallest amount of shared information — a shared risk signal, for instance — needed to prevent this without slowing bots down?*

12. How would you catch a correlated pile-up like this without adding a central observer, which would compromise the pattern?

Answer:
Use distributed risk monitoring. Each bot watches its own exposure and shares lightweight risk indicators with other bots through a shared signal or distributed state.
If multiple bots detect increasing exposure to the same market condition, they can automatically reduce position sizes or pause new trades.

This preserves the Peer/Swarm pattern because no single bot acts as a central controller. Risk awareness emerges from shared signals rather than centralized oversight.

13. What is the smallest amount of shared information needed to prevent this without slowing bots down?

Answer:
A simple shared risk signal is sufficient.

Examples:
- Current aggregate exposure level
- Risk score for a market sector
- Volatility alert indicator
- Position concentration percentage

Bots do not need detailed trade information from every other bot. They only need a minimal shared indicator that warns when collective risk is becoming too high.

Key idea:
Share only enough information to influence local decisions while keeping decision-making decentralized and fast.

Case Study — An Equity Research Desk

Industry	Investment banking — equity research producing a stock recommendation
Pattern	Federated Teams — independent specialist desks collaborate without one overriding another
Core tension	Depth of independent expertise versus one coherent recommendation

The Scenario

A bank needs an official recommendation on a stock — buy, hold, or sell. Three independent desks analyze it through their own lens: Equity Research (valuation and earnings), Credit Research (the company's debt health), and Macro Research (sector and economic trends). No single agent manages all three. Each desk works independently, and a lightweight Coordination layer combines their views into one investment note — flagging any disagreement rather than silently overriding it.

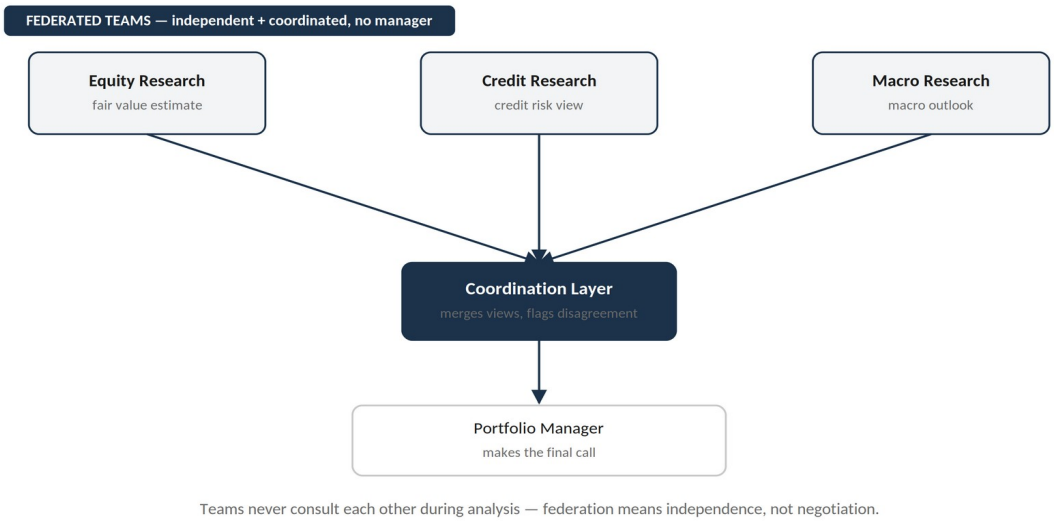


Figure 3.3 — Federated Teams: independent teams contribute in parallel; a coordinator merges views and flags disagreement.

Agent Roster

Team	Internal Composition	Contributes
Equity Research Team	Valuation-model agent and earnings-analysis agent	A fair value estimate
Credit Research Team	Debt-analysis agent and covenant-check agent	A credit risk view
Macro Research Team	Sector-trend agent and rates-sensitivity agent	A macro outlook
Coordination Layer	A lightweight synthesis agent, not a Manager	One merged note, with disagreement flagged

How It Works, Step by Step

1. The same company data is sent to all three desks in parallel — none waits on another.
2. Each desk runs its own internal process, which may itself be Manager–Worker or Sequential, and produces one desk-level view.
3. Desks don't consult each other during analysis — federation means independence, not negotiation.
4. The Coordination layer receives all three views: where they agree, it merges cleanly; where they disagree, it flags the disagreement rather than picking a winner.
5. A human portfolio manager receives the combined note with any disagreement clearly marked, and makes the final call.

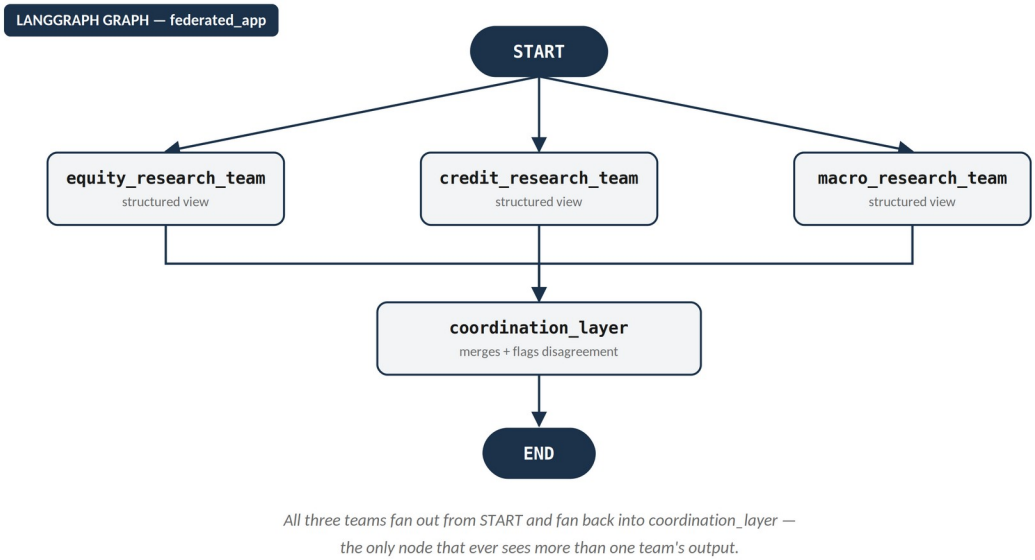


Figure 3.3b — The LangGraph implementation: three teams fan out from START and fan back into coordination_layer — the only node that sees more than one team's output.

> **WHY IT FITS**
> Credit risk, valuation, and macro trends genuinely need different expertise — folding them into one simplified view would lose signal. Federation keeps each desk's full reasoning intact, and disagreement between desks is often the most useful part of the note.

> **WHERE IT BREAKS**
> This is the most expensive pattern here — three full research processes run even for a routine update. The Coordination layer's synthesis logic is hard to calibrate: too aggressive and it quietly resolves disagreements that should reach the portfolio manager; too passive and it just forwards three raw opinions with no real synthesis.

Questions to Think About

14. *How is this different from three Manager–Worker pipelines reporting to one final Manager? Where does federation really start?*
15. *Should the Coordination layer ever resolve a disagreement automatically? What would that condition look like?*
16. *What is the smallest decision size that would justify running this expensive a process?*

PATTERN — SEQUENTIAL PIPELINE

Case Study 4 — Automated Equity Research Report Generation

Industry	Financial research — a weekly sector research report
Pattern	Sequential Pipeline — a strict linear hand-off, with no parallel work
Core tension	Quality control versus total time to finish

The Scenario

A research team wants a weekly sector report generated end to end by agents. The task has a naturally linear shape: you can't analyze filings you haven't gathered, and you can't review a report that hasn't been written yet.

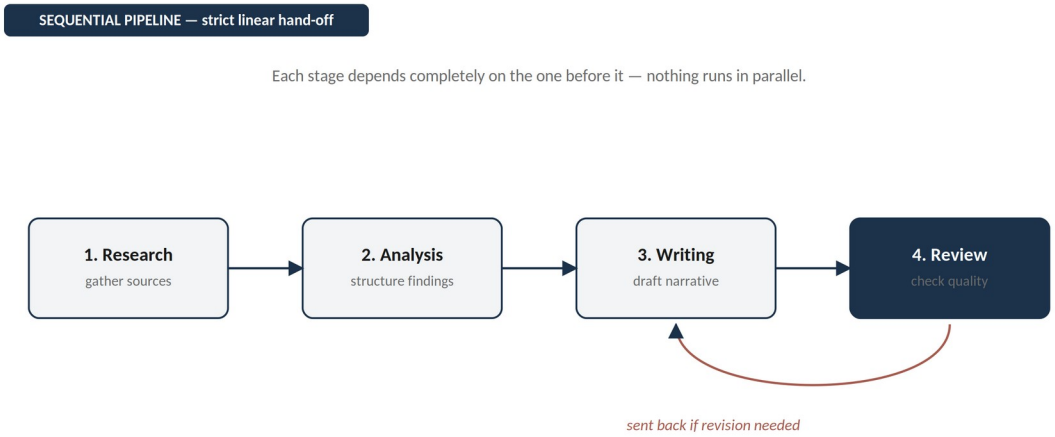


Figure 3.4 — Sequential Pipeline: a strict linear hand-off, with feedback only one step back.

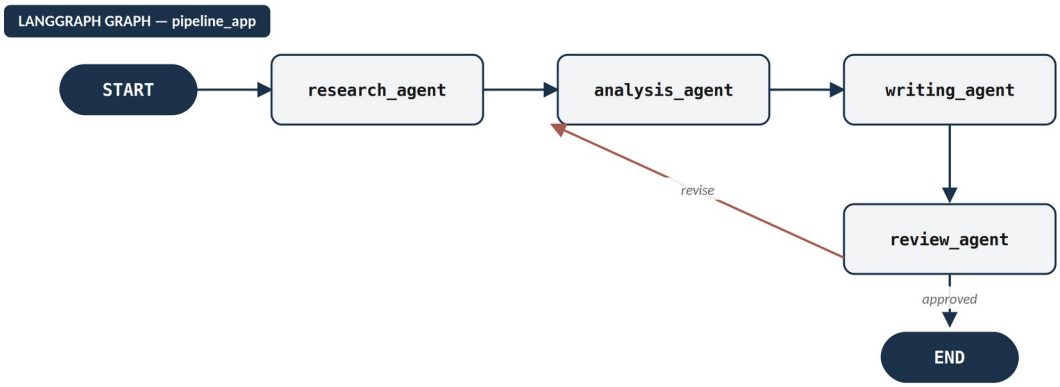
Agent Roster

Stage	Agent	Input	Output
1. Research	Research Agent	Companies to cover, source list	Filings, transcripts, news, citations
2. Analysis	Analysis Agent	Raw findings	Structured insights, trends, comparisons
3. Writing	Writing Agent	Structured insights	Draft report
4. Review	Review Agent	Draft report	Final approved report, or a revision request

How It Works, Step by Step

1. The Research Agent gathers filings, earnings transcripts, and news; nothing downstream starts until this stage finishes.
2. The Analysis Agent receives only the Research Agent's output, not the original raw sources, and produces structured insights.
3. The Writing Agent receives only the structured insights and drafts the report narrative.

4. The Review Agent checks the draft against quality criteria. If it fails, it goes back to the Writing stage — not all the way back to Research — with specific feedback.
5. Once approved, the Review Agent's output is the final report, with no further hand-off.



review_agent routes back exactly one step to writing_agent — never all the way to research_agent.

Figure 3.4b — The LangGraph implementation: review_agent routes back exactly one step to writing_agent, never all the way to research_agent.

WHY IT FITS

The task is genuinely order-dependent — gathering, analyzing, writing, and reviewing can't be run in parallel without producing an incoherent report. Sequential pipelines are also the easiest pattern to log, checkpoint, and resume, which matters for an unattended weekly job.

WHERE IT BREAKS

Total time is the sum of every stage — one slow stage delays the whole report, with no parallel work to absorb that cost. A weak Research stage quietly carries through to Review, where it's most expensive to fix.

Questions to Think About

17. *Where would you add an early check — right after Research, for instance — to avoid discovering a weak input only at Review?*
18. *Could any two stages here run in parallel without hurting quality? Why or why not?*
19. *How does the Review-to-Writing feedback loop make this feel less like a one-way pipeline and more like Planning or ReAct?*

17. Where would you add an early check to avoid discovering a weak input only at Review?
Add a validation/checkpoint stage immediately after the Research Agent.
The check should verify:
- Source completeness
- Citation quality
- Coverage of all required companies
- Sufficient evidence for downstream analysis
This catches poor or missing research before Analysis and Writing consume it, reducing costly rework later in the pipeline.

18. Could any two stages here run in parallel without hurting quality? Why or why not?
Generally, no. Each stage depends on the output of the previous stage:
- Analysis requires completed research.
- Writing requires completed analysis.
- Review requires a completed draft.
Running stages in parallel would risk incomplete context, inconsistencies, or weaker output quality.
Limited parallelism may exist within a stage (e.g., gathering data from multiple companies simultaneously), but the major pipeline stages are naturally sequential.

19. How does the Review-to-Writing feedback loop make this feel less like a one-way pipeline and more like Planning or ReAct?
The feedback loop introduces iteration instead of strict one-way progression.
Rather than simply moving forward:
```
Research → Analysis → Writing → Review → End
```
The process becomes:
```
Research → Analysis → Writing → Review
                        ↑         ↓
                        ←---------
```
The Review Agent evaluates results and provides corrective feedback, allowing the Writing Agent to adjust and improve the draft. This resembles Planning or ReAct patterns because the system can reflect
Key idea:
The workflow is still primarily sequential, but the Review-to-Writing loop adds a controlled form of reasoning, feedback, and refinement.

4. Decision Frameworks — Two Decision Trees

These two trees are used one after another. The first — general agentic pattern selection — gets you to the point of deciding whether you need Multi-Agent at all. The second — the Multi-Agent sub-pattern tree — picks which of the four patterns from Section 3 fits, once you're past that point.

Tree 1 — Choosing a General Agentic Pattern

Follow this sequence of questions for any task:

1. Is the solution path known in advance? If not, does the task need tools or outside data? If yes, add tool use. If not, move to the structure question below.
2. Is this a fixed workflow? If yes, use a Sequential or Structured Workflow — but watch for adding reasoning loops you don't actually need.
3. Can the task's structure be laid out before execution? If yes, use a Planning Pattern, keeping in mind that plans can go stale. If not, use a ReAct Pattern — reasoning and action in a loop, adapting as it goes.
4. Does output quality matter more than speed? If yes, add a Reflection step — generate, critique, refine — which adds time and cost. If not, skip it.
5. Does the task need specialization or scale? If yes, move to a Multi-Agent Architecture and go to Tree 2. If not, a Single Agent is enough.

> **KEY POINT**
> This tree only tells you whether you need agents at all, and which single-agent pattern to start with — it treats Multi-Agent as one single stop. That's exactly what Tree 2 is for.

Tree 2 — Choosing a Multi-Agent Pattern

Once a task lands on Multi-Agent Architecture, use this table to choose among the four Section 3 patterns:

Question	If Yes	If No
Is the task naturally ordered — must step B wait for step A?	Sequential Pipeline	Continue below
Do sub-tasks need deep, independent expertise where disagreement matters?	Federated Teams	Continue below
Is there a natural single owner who can split up and delegate bounded sub-tasks?	Manager and Worker	Continue below
Is the environment fast-moving or spread out, with no natural single owner?	Peer and Swarm	Reconsider whether Multi-Agent is needed at all

> **CHECK YOUR UNDERSTANDING**
> Try mapping each case study back to this table: Support Triage lands on Manager and Worker, Trading Across Exchanges lands on Peer and Swarm, the Equity Research Desk lands on Federated Teams, and the Research Report lands on Sequential. Every case study in this handbook should land on exactly one row.

5. Practice Exercise — Design a Robo-Advisory Platform

FORMAT **Work through this on your own, or with a study partner**

The Scenario

You're designing the multi-agent system behind a robo-advisory investment platform. It must onboard a new client (identity checks and a risk questionnaire), build a portfolio allocation across asset classes, rebalance it periodically, and communicate with the client at every stage — with a human financial advisor always able to override any automated allocation.

Try This

1. Run Tree 1 on the overall platform, then again on two or three individual pieces — client onboarding alone, for instance, may not need Multi-Agent even if the full platform does.
2. For any piece that does need Multi-Agent, run Tree 2 to choose a pattern for it.
3. Decide the top-level architecture — Centralized, Decentralized, or Hybrid — and justify it against Section 2's case studies.
4. Sketch the agent roster: name each agent, its role, and which pattern or architecture layer it belongs to.
5. Find at least one point where a human advisor must be able to step in, and explain why the system can't fully automate past that point.

If You Get Stuck

- Is onboarding a single client the same shape of problem as running the whole platform? Could it be handled by a single agent?
- Portfolio rebalancing touches market data and client risk limits at the same time — does that pull toward Decentralized, or can one Manager still own it?
- Would Federated Teams make sense if the platform offers more than one strategy — for example, a growth strategy and an income strategy, each researched independently?
- What part of this system absolutely has to happen in order, with no parallel work possible?

One Way to Approach This

A common design lands on a Hybrid architecture overall, with a central Advisory Manager owning the client relationship. Onboarding and risk profiling use Manager and Worker, since a single agent can cleanly delegate identity checks and questionnaire scoring. Portfolio construction runs as a Sequential Pipeline — gather data, run the risk model, allocate, run a compliance check — since each step depends on the one before it. If the platform offers multiple strategies, Federated Teams lets each strategy's research stay independent before a Coordination layer combines them into one recommendation for the advisor. This closely mirrors Case Study 3's shape — worth comparing side by side.

6. Summary and Key Takeaways

Architecture — where control lives

- Centralized: one Manager holds full control. Simple to audit, but a single point of failure. (Case Study 1)
- Decentralized: no central controller; agents act independently. Fast and resilient, but hard to see the whole picture. (Case Study 2)
- Hybrid: a central owner for the outward-facing decision, with decentralized teams doing the internal work. Most production systems land here. (Case Study 3)

Design Pattern — how agents interact

- Manager and Worker: bounded request and response, no Worker-to-Worker talk.
- Peer and Swarm: symmetric, independent action with no fixed roles.
- Federated Teams: independent specialist teams, combined by a lightweight coordinator that preserves disagreement.
- Sequential Pipeline: strict linear hand-off, easiest to log and resume, but total time is the sum of every stage.

Two Questions Worth Asking on Any New Task

First: do I need Multi-Agent at all, or would a single agent — possibly with tools, planning, or reflection — already do the job? Second, if Multi-Agent is the right call: is this task ordered, does it need independent expertise, does it have a natural owner, or is it fast-moving with no natural owner? That second question is what picks the pattern.

What's Next

The Practice Exercise in Section 5 is exactly the kind of design that gets built as running code in LangGraph and CrewAI — same agents, same architecture decision, now implemented rather than sketched.